

面向蛋白质图分类的交叉残差图神经网络

付小琴*

摘要

研究背景。现有图神经网络通常使用跳跃连接来稳定深层消息传递，但在面向蛋白质的图分类中，它们很少直接隔离如下问题：跨分支交换或图摘要重新注入，是否优于普通的同路径残差复用。

研究方法。本文提出交叉残差图神经网络（CR-GNN），作为一个受控比较框架，覆盖五种信息流拓扑：普通堆叠、节点级残差复用、节点级交叉交换、图级残差传播和图级交叉摘要交换。所有变体均在共享主干和分层五折验证下评估。主要证据来自 PROTEINS 和 DD 这一蛋白质导向主对比；ENZYMES 被视为支持性压力测试，MUTAG、AIDS 和 Mutagenicity 则作为补充鲁棒性数据集。针对性消融和深度扩展分析用于解释 benchmark 中出现的模式。

研究结果。没有任何单一复用拓扑能够全面占优。在主要蛋白质导向比较中，NodeResGNN 在 PROTEINS 上最佳，GraphResGNN 在 DD 上最佳。ENZYMES 也指向 GraphResGNN 更强，但由于其绝对准确率较低，更适合作为压力测试而非主要 biological evidence。交叉残差变体不是普适赢家，但在 DD 上保持竞争力，并在针对性消融中与稀疏路由表现出较好配合。在 24 个数据集-算子组合中，GraphResGNN 获胜最多（12/24）。基于 PROTEINS 的深度扩展分析进一步揭示了一个 CKA-性能悖论：GraphResGNN 具有最低的深层 CKA，却取得最多胜场；NodeResGNN 拥有最高 CKA，却获胜较少。

研究结论。在面向蛋白质的图分类中，残差设计具有明显的数据集依赖性。图级残差传播是当前 benchmark 中最强的默认选择，但并非在所有场景都最佳；节点级残差复用在部分数据集上仍然最强，而交叉残差交换更适合被理解作为一种选择性替代方案，而非普适替换。更强的层间连续性同样不必然意味着更好的最终准确率。

目录

1 引言	3
1.1 主要贡献	4

*柳州铁道职业技术学院铁路机车与车辆学院，广西柳州 545000

目录	2
2 材料与方法	5
2.1 问题定义与符号	5
2.2 共享消息传递与图级学习	5
2.3 CR-GNN 架构家族	6
2.3.1 单分支与节点级残差变体	6
2.3.2 图级残差传播	8
2.4 残差路由机制	8
2.5 数据集	10
2.6 训练配置与实现细节	11
2.7 实验设计	11
3 结果	12
3.1 整体 benchmark 表现	13
3.2 Biological core 与交叉残差优势	13
3.3 补充鲁棒性参考	14
3.4 机制：表示连续性与性能的张力	14
3.5 量化这些权衡	16
3.6 消融证据	17
3.6.1 Gate ablation	17
3.6.2 残差路由	18
3.6.3 统计检验	18
3.6.4 参数扫描	18
3.7 探索性敏感性分析	18
4 讨论	19
4.1 RQ1: 为什么表示连续性最弱的架构反而最常获胜?	19
4.2 RQ1 & RQ2: 不同复用拓扑何时占优?	19
4.3 RQ3: 路由策略是否在架构选择之外仍然重要?	20
4.4 局限性	21
4.5 未来方向	22
5 结论	22
6 致谢	22
A 附录	23
A.1 补充鲁棒性 benchmark	23
A.2 补充统计检验	23

1 引言

图神经网络 (GNN) 已成为分子图和蛋白质图学习的标准方法, 因为它们能够同时编码局部邻域结构与更高阶的关系上下文 (Kipf and Welling, 2016; Gilmer et al., 2017; Hamilton et al., 2017; Wu et al., 2021; Zhou et al., 2020)。在生物分子图分类中, 这种组合尤为重要, 因为判别信号往往同时依赖局部生化基序和更广泛的图级组织。本文聚焦于以 PROTEINS 和 DD 为核心的蛋白质导向 benchmark 设定, 将 ENZYMES 作为支持性的生物压力测试, 并将 MUTAG、AIDS 和 Mutagenicity 视为补充鲁棒性数据集, 而非与前者同等核心的生物证据。因此, 本文的目标不是为下游生物发现提供直接验证, 而是在蛋白质相关图分类设定下研究残差路由行为。

这一设定下的主要方法论困难在于, 更深的消息传递并不会自动带来收益。重复传播可能因过平滑及相关深度效应抑制有信息量的中间状态 (Li et al., 2018; Oono and Suzuki, 2020; Chen et al., 2020b), 而过压缩则可能将远距离节点信息挤压进无效瓶颈 (Alon and Yahav, 2021; Topping et al., 2021)。残差连接、归一化与正则化虽能缓解部分退化 (He et al., 2016; Bresson and Laurent, 2017; Zhao and Akoglu, 2020; Cai et al., 2021; Rong et al., 2019; Chen et al., 2020a), 但大多数残差设计仍沿同一路径复用信息, 因此它们改善了优化过程, 却未直接检验替代性信息流拓扑是否能保留更有用的历史信号。

相关图学习工作已经探索了跳跃连接、稠密聚合、APPNP 式传播、Jumping Knowledge 读出以及层次化池化 (Xu et al., 2018; Klicpera et al., 2018; Gao and Ji, 2019; Ying et al., 2018)。这些方法能够保留来自不同深度或不同池化层级的信息, 但并未直接隔离如下问题: 跨分支交换隐藏状态, 或跨块边界重新注入池化后的图摘要, 是否优于普通的同路径残差复用。此前尚无工作在受控协议下直接比较这些复用拓扑, 这正是本文要解决的问题。

本文研究交叉残差图神经网络 (CR-GNN), 这是一个受控模型家族, 在共享图分类主干下比较普通堆叠、节点级残差复用、节点级交叉残差交换、图级残差传播和图级交叉摘要交换。面向论文主叙事的证据主要来自蛋白质导向核心, 尤其是 PROTEINS 和 DD; 非蛋白质数据集仅用于检验这些结构偏好在核心之外是否仍然稳定。研究围绕三个问题展开:

1. **为什么表示连续性最弱的架构反而最常获胜?** 如果残差复用通常被理解为跨深度保留有效信息的手段, 那么当表现最好的架构反而展现出最大的层间表示变化时, 这意味着什么?
2. **不同复用拓扑何时占优?** 在以 PROTEINS 和 DD 为核心、辅以支持性压力测试和补充鲁棒性数据集的比较中, 何时节点级残差复用更强, 何时图级残差传播更优, 何时交叉残差交换会成为有竞争力甚至更合适的选择?
3. **残差路由设计会如何改变答案?** 超越“是否存在额外复用路径”这一二元问题, 不

同过滤策略，例如可学习的稠密路由、top-k 选择和稀疏抑制，将如何影响残差架构与交叉残差架构的相对表现？

围绕这三个问题组织本文，对界定方法学范围非常关键。本文并不主张某一条额外通路总能带来更好的 GNN，而是试图揭示残差路由在何时、以何种方式有效；普通残差复用何时仍然更强；以及在主干、数据划分和训练协议固定后，为什么层间连续性并不会单调映射到最终准确率。

本文可视为一项面向蛋白质导向生物分子图分类的方法学研究。主要证据来自 PROTEINS 和 DD，ENZYMES 被保留为更困难的支持性生物 benchmark，其较低的绝对准确率本身也有助于刻画任务难度。MUTAG、AIDS 和 Mutagenicity 用于测试蛋白质核心之外的鲁棒性，但它们并不定义本文的主要 biological claim。这种设计使论文保持在具有生物意义的图设定附近，同时仍具备可复现且受控的 benchmark 范围。

1.1 主要贡献

本文的主要贡献如下：

- **提出一个用于蛋白质导向图分类复用拓扑比较的受控框架。**在相同主干、相同数据划分和相同训练协议下，比较五种信息流设计：普通堆叠、节点级残差复用、节点级交叉残差交换、图级残差传播和图级交叉摘要交换。通过固定其他因素，该框架隔离了复用拓扑对性能、一致性和表示动态的影响，其中最强的 biological emphasis 明确落在 PROTEINS 和 DD 上。
- **识别出峰值准确率、排序稳定性和表示连续性之间的系统性权衡。**没有单一拓扑能同时支配这三个维度。在当前 benchmark 中，图级残差传播最常获得最高峰值准确率，但排序波动明显；交叉残差变体具有竞争力且通常较稳定，但并非普适赢家；而最强的表示连续性并不能预测最佳分类准确率。借助 winner count 和 rank stability 指标量化这种三方张力，为 GNN 架构评价提供了超越单一榜单数字的新视角。
- **给出揭示 CKA-性能悖论的深度扩展机制分析。**在不同深度上跟踪逐层 CKA、余弦相似度和梯度范数，结果表明表示连续性最弱的架构反而取得了最多获胜次数，这挑战了“更强的层间稳定性意味着更好性能”的假设。这说明当表示变化以结构化方式发生并伴随适当的上下文重新注入时，它可能是有益而非破坏性的。
- **证明路由策略是一个有意义的次级设计轴。**轻度稀疏路由在代表性目标上最常获胜，尤其常与交叉导向设定配对；而可学习的稠密路由仍在最强的图级残差目标上保持优势。这使讨论从二元架构选择扩展为“拓扑 + 路由”的联合设计问题。

2 材料与方法

本节定义统一的图分类设定、CR-GNN 架构家族、benchmark 套件以及共享实验协议。核心原则是在尽量保持其余流程一致的前提下，隔离“信息在何处被复用”。这一设计对论文的方法论论证至关重要：如果主干算子、池化方式和评估协议都被固定，那么性能差异就更可能来源于复用拓扑本身，而不是无关的实现差异。

2.1 问题定义与符号

设 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 为无向或有向图，其中 $\mathcal{V} = \{v_1, v_2, \dots, v_n\}$ 是包含 n 个节点的集合， $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ 是边集。每个节点 $v_i \in \mathcal{V}$ 关联一个特征向量 $\mathbf{x}_i \in \mathbb{R}^{d_{\text{in}}}$ ，整图的节点特征记为 $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ ，图结构由邻接矩阵 $\mathbf{A}$ 表示。

给定训练数据集 $\mathcal{D} = \{(\mathcal{G}_i, y_i)\}_{i=1}^N$ ，目标是学习映射 $f_\theta : \mathcal{G} \rightarrow \hat{y}$ ，用于预测未见图的标签。本文在同一图分类流程下比较五种架构：

- **PlainGNN**：无显式状态复用的顺序消息传递
- **NodeResGNN**：单分支节点级残差复用
- **NodeCrossGNN**：双分支节点级交叉交换
- **GraphResGNN**：跨顺序块的图级残差传播
- **GraphCrossGNN**：跨成对块的图级交叉摘要交换

这组比较旨在隔离“信息在哪里被复用”：是在单分支内部、在并行分支之间，还是通过块间传递的池化图摘要。对应地，同分支残差复用主要检验重新注入历史状态能否稳定更深层传播，而交叉残差设计则检验跨并行流交换信息能否保留普通堆叠中可能丢失的互补结构。

2.2 共享消息传递与图级学习

设 Φ 表示一个消息传递算子。对于给定主干，节点表示经过 L 层更新为

$$\mathbf{h}_i^{(\ell+1)} = \sigma \left(\mathbf{W}^{(\ell)} \cdot \text{AGGREGATE}_\Phi \left(\left\{ \mathbf{h}_j^{(\ell)} : j \in \mathcal{N}(i) \right\}, \mathbf{h}_i^{(\ell)} \right) + \mathbf{b}^{(\ell)} \right). \quad (1)$$

经过 L 层后，读出函数 R 将节点级表示聚合为图嵌入

$$\mathbf{h}_\mathcal{G} = R \left(\left\{ \mathbf{h}_i^{(L)} : i = 1, \dots, n \right\} \right). \quad (2)$$

分类器输出为

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{W}_{\text{clf}} \mathbf{h}_\mathcal{G} + \mathbf{b}_{\text{clf}}), \quad (3)$$

模型通过最小化交叉熵进行端到端训练：

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}). \quad (4)$$

2.3 CR-GNN 架构家族

CR-GNN 被定义为一个紧凑的图分类架构家族。所有变体共享相同的输入投影、堆叠消息传递层、图级池化和线性分类器；它们只在中间节点状态与图级摘要如何跨深度和跨分支复用上有所不同。这里特意不引入独立编码器、任务特定特征工程或架构特定读出头，因为这些额外组件会使人难以判断性能增益究竟来自残差拓扑还是额外模型容量。

设 $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ 为一个图，其节点特征矩阵为 $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{in}}}$ ，邻接结构为 $\mathbf{A}$ 。模型可实例化为四种标准消息传递算子之一：

$$\Phi \in \{\text{GCNConv}, \text{GATConv}, \text{SAGEConv}, \text{GINConv}\},$$

并在单个模型实例内始终复用同一种算子。设隐藏宽度为 d ，每个分支都从输入投影开始：

$$\mathbf{H}^{(0)} = \psi(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (5)$$

随后经过 L 个隐藏层和一个读出：

$$\mathbf{g} = \text{Pool}(\mathbf{H}^{(L)}). \quad (6)$$

其中， $\Phi_{\text{in}} : \mathbb{R}^{n \times d_{\text{in}}} \rightarrow \mathbb{R}^{n \times d}$ 表示分支特定的输入算子， $\text{Pool}(\cdot)$ 表示对节点执行全局均值池化，而

$$\psi(\mathbf{Z}) = \text{Dropout}(\text{ReLU}(\mathbf{Z})). \quad (7)$$

残差信号由标量门控系数 $\alpha \in (0, 1]$ 调制，默认通过 sigmoid 参数化学习得到，仅在 gate ablation 中固定。残差路由算子记为

$$\mathcal{R}(\mathbf{U}) = \begin{cases} \mathbf{U}, & \text{可学习稠密路由,} \\ \mathbf{U} \odot \mathbf{M}_k, & \text{top-k 路由,} \\ \text{softshrink}_{\lambda}(\mathbf{U}), & \text{稀疏路由,} \end{cases} \quad (8)$$

其中 $\mathbf{M}_k$ 沿特征维保留绝对值最大的通道， λ 为稀疏路由阈值。

2.3.1 单分支与节点级残差变体

PlainGNN. 普通基线不做残差复用，而是直接堆叠 L 层相同算子：

$$\mathbf{H}^{(\ell+1)} = \psi(\Phi_{\ell}(\mathbf{H}^{(\ell)}, \mathbf{A})), \quad \ell = 0, \dots, L-1. \quad (9)$$

NodeResGNN. 第一种残差变体在同一分支内保存前一层隐藏状态。为与实现一致，第一步残差输入初始化为零：

$$\mathbf{P}^{(0)} = \mathbf{0}, \quad \mathbf{P}^{(\ell+1)} = \mathbf{H}^{(\ell)}. \quad (10)$$

其更新写为

$$\mathbf{H}^{(\ell+1)} = \psi\left(\Phi_\ell\left(\mathbf{H}^{(\ell)} + \alpha \mathcal{R}\left(\mathbf{P}^{(\ell)}\right), \mathbf{A}\right)\right), \quad \ell = 0, \dots, L-1. \quad (11)$$

NodeCrossGNN. 节点级交叉残差模型维护两个具有独立输入投影的并行分支：

$$\mathbf{H}_1^{(0)} = \psi\left(\Phi_{\text{in}}^{(1)}(\mathbf{X}, \mathbf{A})\right), \quad \mathbf{H}_2^{(0)} = \psi\left(\Phi_{\text{in}}^{(2)}(\mathbf{X}, \mathbf{A})\right), \quad (12)$$

交换状态初始化为

$$\mathbf{P}_1^{(0)} = \mathbf{0}, \quad \mathbf{P}_2^{(0)} = \mathbf{0}, \quad \mathbf{P}_b^{(\ell+1)} = \mathbf{H}_b^{(\ell)}. \quad (13)$$

每个分支接收另一分支的前一时刻状态：

$$\mathbf{H}_1^{(\ell+1)} = \psi\left(\Phi_\ell^{(1)}\left(\mathbf{H}_1^{(\ell)} + \alpha \mathcal{R}\left(\mathbf{P}_2^{(\ell)}\right), \mathbf{A}\right)\right), \quad (14)$$

$$\mathbf{H}_2^{(\ell+1)} = \psi\left(\Phi_\ell^{(2)}\left(\mathbf{H}_2^{(\ell)} + \alpha \mathcal{R}\left(\mathbf{P}_1^{(\ell)}\right), \mathbf{A}\right)\right), \quad \ell = 0, \dots, L-1. \quad (15)$$

最终节点表示为两分支求和：

$$\mathbf{H}^{(L)} = \mathbf{H}_1^{(L)} + \mathbf{H}_2^{(L)}. \quad (16)$$

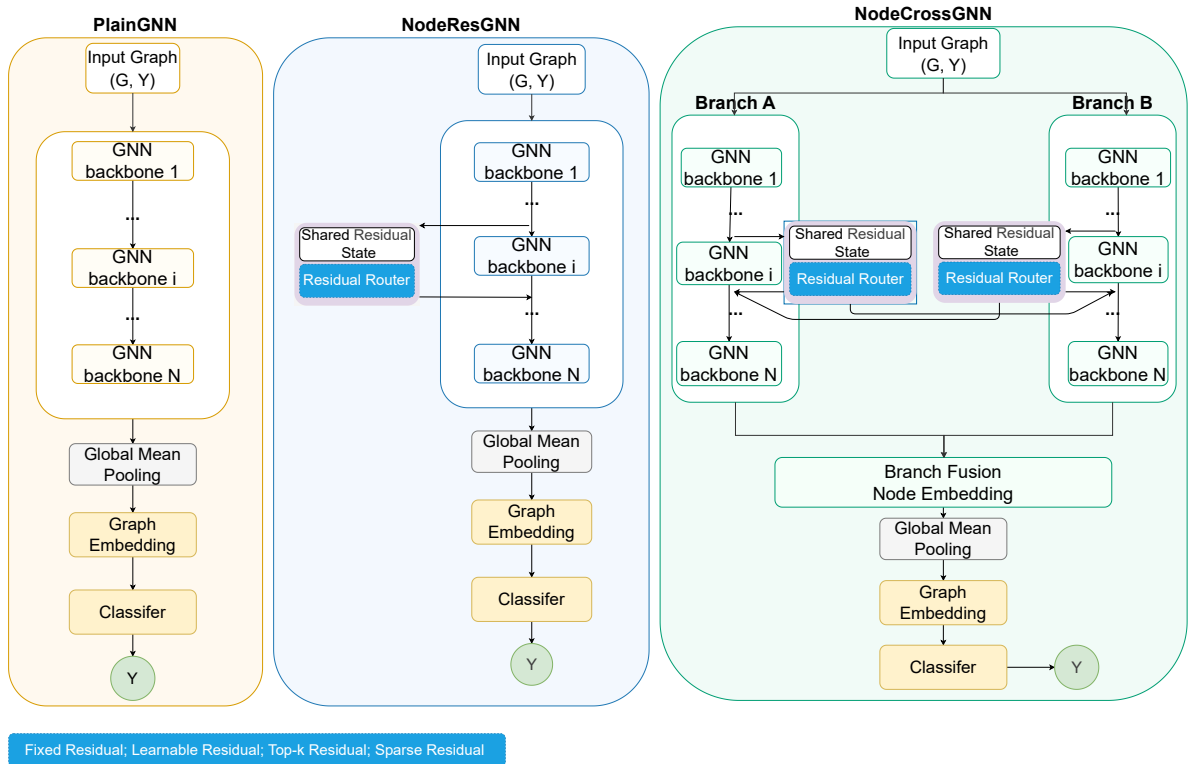


图 1: 节点级 CR-GNN 架构家族。左: PlainGNN, 无状态复用的顺序消息传递。中: NodeResGNN, 单分支残差复用, 每层接收前一隐藏状态作为附加输入。右: NodeCrossGNN, 双分支交叉残差交换, 每个分支接收对侧分支的先前隐藏状态; 两分支在全局均值池化与分类前求和。

2.3.2 图级残差传播

GraphResGNN. 该变体串联三个图条件块。设 $\mathcal{B}(\mathbf{X}, \mathbf{A}, \mathbf{g})$ 表示一个图条件块。块内部将图级状态 $\mathbf{g} \in \mathbb{R}^{B \times d}$ 通过 batch 索引广播回节点，并在每个隐藏消息传递层前进行相加：

$$\mathbf{Z}^{(0)} = \psi(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (17)$$

$$\mathbf{Z}^{(\ell+1)} = \psi(\Phi_{\ell}(\mathbf{Z}^{(\ell)} + \alpha \mathcal{R}(\mathbf{g}[\text{batch}]), \mathbf{A})), \quad \ell = 0, \dots, L-1, \quad (18)$$

$$\mathcal{B}(\mathbf{X}, \mathbf{A}, \mathbf{g}) = \text{Pool}(\mathbf{Z}^{(L)}) + \alpha \mathcal{R}(\mathbf{g}). \quad (19)$$

若不存在上一阶段的图状态，则取 $\mathbf{g} = \mathbf{0}$ 。于是 GraphResGNN 的递推为

$$\mathbf{g}^{(0)} = \mathbf{0}, \quad \mathbf{g}^{(t)} = \mathcal{B}_t(\mathbf{X}, \mathbf{A}, \mathbf{g}^{(t-1)}), \quad t = 1, 2, 3, \quad (20)$$

并使用 $\mathbf{g}^{(3)}$ 进行分类。

GraphCrossGNN. 四个块被组织为两个顺序阶段。在该模型中，图级交换仅发生在池化后的图嵌入阶段，而不直接进入节点更新。设 $\mathcal{C}(\mathbf{X}, \mathbf{A}, \mathbf{g})$ 表示一个此类块：

$$\mathbf{Z}^{(0)} = \psi(\Phi_{\text{in}}(\mathbf{X}, \mathbf{A})), \quad (21)$$

$$\mathbf{Z}^{(\ell+1)} = \psi(\Phi_{\ell}(\mathbf{Z}^{(\ell)}, \mathbf{A})), \quad \ell = 0, \dots, L-1, \quad (22)$$

$$\mathcal{C}(\mathbf{X}, \mathbf{A}, \mathbf{g}) = \text{Pool}(\mathbf{Z}^{(L)}) + \alpha \mathcal{R}(\mathbf{g}). \quad (23)$$

设两个分支状态初始化为 $\mathbf{g}_1^{(0)} = \mathbf{g}_2^{(0)} = \mathbf{0}$ ，则分阶段递推为

$$\hat{\mathbf{g}}_1^{(t)} = \mathcal{C}_{2t-1}(\mathbf{X}, \mathbf{A}, \mathbf{g}_1^{(t-1)}), \quad (24)$$

$$\hat{\mathbf{g}}_2^{(t)} = \mathcal{C}_{2t}(\mathbf{X}, \mathbf{A}, \mathbf{g}_2^{(t-1)}), \quad t = 1, 2, \quad (25)$$

$$\mathbf{g}_1^{(t)} = \hat{\mathbf{g}}_2^{(t)}, \quad \mathbf{g}_2^{(t)} = \hat{\mathbf{g}}_1^{(t)}. \quad (26)$$

最终图嵌入为

$$\mathbf{g}_{\text{final}} = \mathbf{g}_1^{(2)} + \mathbf{g}_2^{(2)}. \quad (27)$$

2.4 残差路由机制

除架构家族本身外，本文还评估残差信息“如何”被注入。因为仅靠架构比较，无法回答表面上的残差优势究竟来自额外路径的存在，还是来自这一路径被过滤的方式。针对性消融中使用三种路由模式：

- **可学习路由：**由自适应门控调制的稠密残差注入。
- **Top-k 路由：**仅保留最强残差通道后再注入。
- **稀疏路由：**抑制弱残差系数，仅保留经过筛选的残差路径。

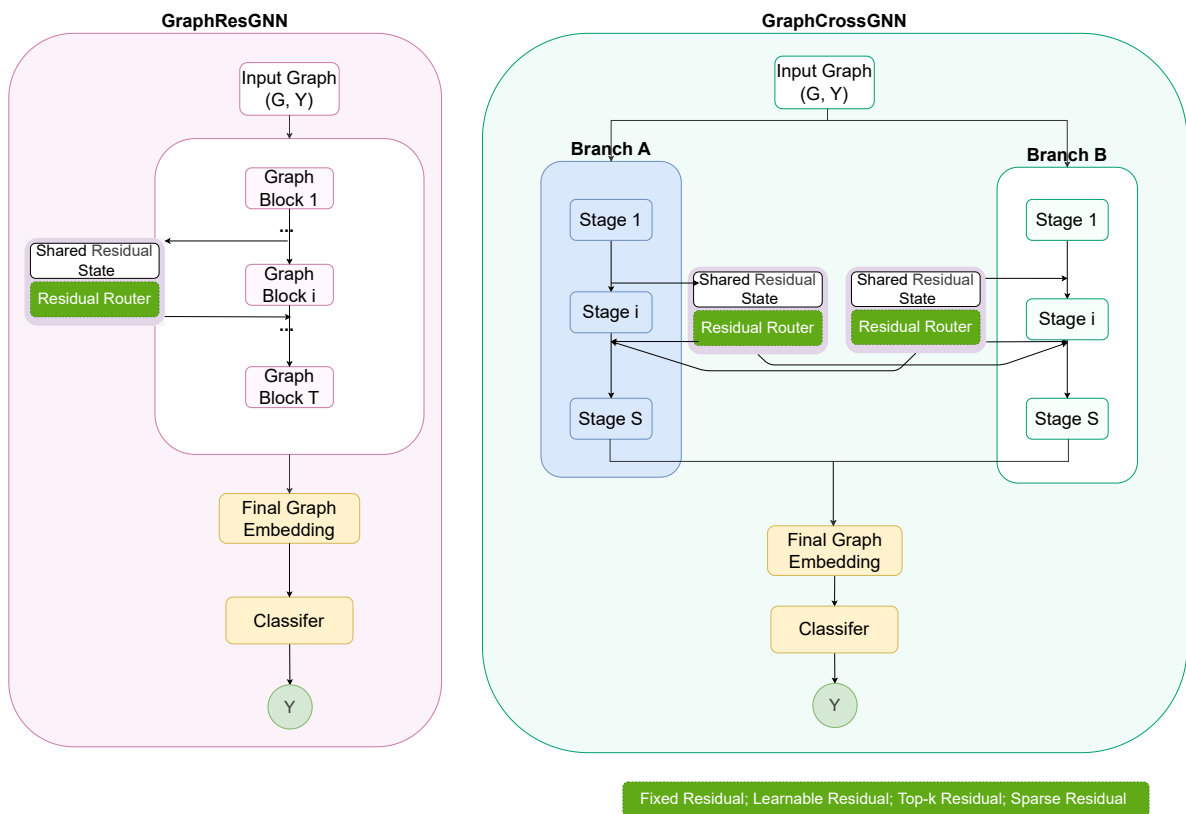


图 2: 图级 CR-GNN 架构家族。左: GraphResGNN, 顺序图条件块之间通过残差路由器传播共享残差状态。右: GraphCrossGNN, 双分支交叉摘要交换, 两条并行块序列在成对阶段之间交换图级摘要。残差路由器支持固定、可学习、top-k 和稀疏四种模式。

这些路由模式并不用于重新定义核心架构家族，而是在实验设计中叠加于部分残差与交叉残差目标之上的机制层。因此，路由研究是解释性的而非装饰性的：它检验同一架构在残差信息被稠密、选择性或稀疏地向前传递时，是否会出现不同表现。

2.5 数据集

为在具有生物意义的设定下检验所提架构，本文将 benchmark 组织为两个层级。

主 biological benchmark 由 PROTEINS、DD 和 ENZYMES 构成 (Fey and Lenssen, 2019; Morris et al., 2020)。补充鲁棒性套件由 MUTAG、AIDS 和 Mutagenicity 构成。这种组织方式使主要 biological claim 聚焦于蛋白质导向数据集，同时仍能在额外分子图上检验结构鲁棒性。

PROTEINS PROTEINS 包含蛋白质的图表示，其中节点表示二级结构元素，边编码它们之间的邻域关系。任务为二分类图分类。该数据集含 1,113 张图，2 个类别，每个节点 3 维输入特征。

DD DD 是一个蛋白质结构数据集，每张图对应一个蛋白质，节点表示氨基酸或与结构相关的单元，边由邻近关系导出。任务为二分类。DD 含 1,178 张图，2 个类别，89 维输入特征。

ENZYMES ENZYMES 是一个 6 类酶分类 benchmark，包含 600 张图和 3 维输入特征。它保留了酶导向的生物解释，同时仍可与本文中的其他图分类 benchmark 直接比较。

所有数据集均通过统一 benchmark 接口访问 (Fey and Lenssen, 2019; Morris et al., 2020)。我们尽量保持轻量预处理，不引入任务特定特征工程、图增强或边重设计。

所有结果均遵循相同的分层两阶段评估设计：

- **外层评估**：图级别的分层五折交叉验证
- **内层验证**：从训练折中抽取验证比例为 0.1 的分层验证子集

主要评估指标是保留测试折上的图分类准确率。对于测试集为 $\mathcal{T}_k$ 的第 k 折，准确率定义为

$$\text{Acc}_k = \frac{1}{|\mathcal{T}_k|} \sum_{(\mathcal{G}_i, y_i) \in \mathcal{T}_k} \mathbf{1}[\hat{y}_i = y_i]. \quad (28)$$

跨五折，我们报告平均准确率

$$\overline{\text{Acc}} = \frac{1}{5} \sum_{k=1}^5 \text{Acc}_k \quad (29)$$

表 1: 主 biological benchmark 数据包的统一概览。

数据集	划分	图数量	类别数	特征维	平均节点/边	角色
PROTEINS	5-fold CV + val	1113	2	3	39.1 / 72.8	主
DD	5-fold CV + val	1178	2	89	284.3 / 715.7	主
ENZYMES	5-fold CV + val	600	6	3	32.6 / 62.1	主

表 2: 保留在核心 biological narrative 之外的补充鲁棒性数据集。

数据集	划分	图数量	类别数	特征维	平均节点/边	角色
MUTAG	5-fold CV + val	188	2	7	17.9 / 19.8	补充
AIDS	5-fold CV + val	2000	2	38	15.7 / 16.2	补充
Mutagenicity	5-fold CV + val	4337	2	14	30.0 / 30.6	补充

及其标准差

$$\sigma_{\text{Acc}} = \sqrt{\frac{1}{5} \sum_{k=1}^5 (\text{Acc}_k - \overline{\text{Acc}})^2}. \quad (30)$$

该 benchmark 在四种消息传递算子 (GCNConv、GATConv、SAGEConv、GINConv) 下评估五种架构。

2.6 训练配置与实现细节

所有模型均在 PyTorch Geometric (Fey and Lenssen, 2019) 中实现, 并遵循统一训练协议。关键超参数总结于表 3。训练采用早停: 若验证损失连续 60 个 epoch 未改善则终止, 并保留验证损失最低的 checkpoint 用于测试评估。不使用 batch normalization 或 layer normalization, 唯一正则化来自 dropout 和 weight decay。所有训练、验证和测试划分均由全局随机种子 1024 固定, 确保每个模型在每一折内看到完全相同的数据划分。不使用任务特定特征工程、图增强或边重设计, 直接采用 TUDataset 的原始节点特征和邻接结构。

残差与交叉残差变体使用的可学习门控, 对一个初始值为 0.8 的标量参数施加 sigmoid 激活, 从而在训练中自适应缩放每次残差注入。对于 top-k 路由, 仅保留残差通道中绝对值最大的那一部分。对于稀疏路由, 使用阈值为 λ 的 softshrink 算子抑制幅值低于 λ 的残差系数, 仅保留筛选后的残差路径。

2.7 实验设计

本实证研究将确认性五折证据与探索性分析区分开来。统一协议下, benchmark 覆盖六个数据集、五种架构、四种消息传递算子和五个外层折。整体 benchmark 被分

表 3: 所有实验共享的关键训练超参数。

参数	数值
框架	PyTorch Geometric
隐藏维度 (d)	64
消息传递层数 (L)	4
Dropout 比例	0.5
优化器	Adam
初始学习率	5×10^{-3}
权重衰减	1×10^{-4}
学习率调度	ReduceLROnPlateau (factor 0.5, patience 15)
最小学习率	1×10^{-5}
Batch size	256
最大训练轮数	200
早停耐心值	60 epochs
梯度裁剪范数 (L_2)	2.0
池化方式	全局均值池化
损失函数	交叉熵
随机种子	1024
验证集比例	0.1

为 biological core (PROTEINS、DD、ENZYMES) 与 supplementary robustness suite (MUTAG、AIDS、Mutagenicity)。

为回答引言中的研究问题，研究被组织为四个实验层次：

1. **完整 benchmark**。在六个数据集和四种算子上，对五种架构进行分层五折比较，以建立整体性能层级，并找出各拓扑最擅长的数据集-算子组合。
2. **深度扩展分析**。在 PROTEINS 的 GCNConv 设定下，跟踪隐藏状态相似度 (CKA 与 cosine) 以及逐层梯度范数，以分析 benchmark 模式背后的表示动态。
3. **针对性消融**。通过 gate ablation、routing ablation 以及参数扫描，检验观察到的差异是否由残差注入机制而非单纯架构标签驱动。
4. **统计检验**。对主要 biological evidence 执行配对 t 检验，以判断观察到的性能差异相对于折间波动是否具有说服力；补充数据集的统计检验放入 appendix。

3 结果

除特别说明外，所有结果均报告为五折 mean best test accuracy $\pm$ 折级标准差；单折敏感性分析仅作探索性参考。

3.1 整体 benchmark 表现

在 GCNConv 设定下, 该 benchmark 对六个数据集和五种 CR-GNN 架构进行了五个外层折评估。对于本文的 paper-facing 论证, 表 4 是主结果表: 它承载以 PROTEINS 和 DD 为核心、并将 ENZYMES 作为支持性生物压力测试的蛋白质导向比较; 表 5 则被保留为补充鲁棒性证据。在主比较对中, NodeResGNN 在 PROTEINS 上最佳 (0.7143), GraphResGNN 在 DD 上最佳 (0.7275)。两者差距通常不大: 在 PROTEINS 上, GraphResGNN 仅落后 NodeResGNN 0.0072; 在 DD 上, 最强的三个模型 GraphResGNN、GraphCrossGNN 和 NodeCrossGNN 均位于 0.0111 准确率点以内。

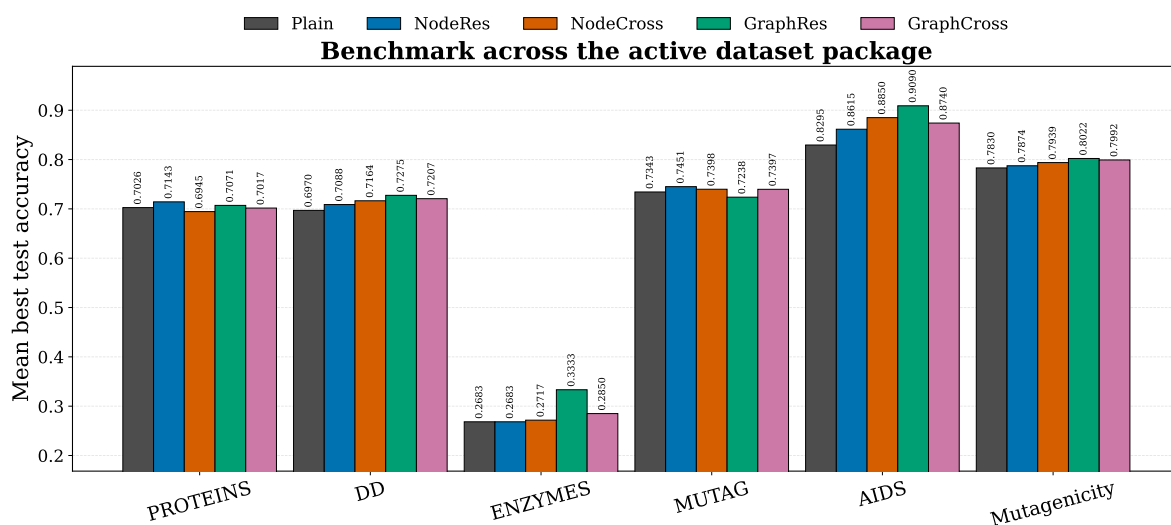


图 3: 在 GCNConv 比较下, 当前有效数据集包上的 V3 全套 benchmark 结果。柱形表示五折 mean best test accuracy, 误差条为标准差。

图 3 将这一模式以可视化方式进一步明确。放眼全套数据, 图级残差传播是更强的默认选择; 但本文的主要 biological claim 应从蛋白质导向核心读取, 而不能由六个数据集的整体平均印象直接替代。

3.2 Biological core 与交叉残差优势

biological core 是本文面向投稿主线的最关键部分, 而在这个核心内部, 真正承载主要生物比较的是 PROTEINS 和 DD。ENZYMES 被保留为支持性压力测试, 因此我们同时考察三者, 但始终保留这种层级关系。

图 4 表明, 这组核心数据并不支持单一的 biological narrative。在 PROTEINS 上, NodeResGNN 最优, GraphResGNN 次之; 在 DD 上, 排序发生转移: GCNConv 主表中 GraphResGNN 最优, 但两个交叉残差模型紧随其后。这两个数据集共同构成主要 biological comparison, 它们支持的是“分裂型优势区间”而非“单一普适赢家”。在 ENZYMES 上, GraphResGNN 再次最优, 但所有模型表现都较差, 最佳平均准确率也

表 4: 主 biological benchmark 结果。单元格为五折 mean $\pm$ std [min, max]。加粗表示该数据集上的最佳均值。

模型	PROTEINS	DD	ENZYMES
PlainGNN	0.7026 $\pm$ 0.0331 [0.655, 0.735]	0.6970 $\pm$ 0.0421 [0.646, 0.763]	0.2683 $\pm$ 0.0509 [0.217, 0.358]
NodeResGNN	0.7143 $\pm$ 0.0223 [0.685, 0.744]	0.7088 $\pm$ 0.0332 [0.651, 0.754]	0.2683 $\pm$ 0.0517 [0.217, 0.367]
NodeCrossGNN	0.6945 $\pm$ 0.0488 [0.613, 0.735]	0.7164 $\pm$ 0.0315 [0.677, 0.771]	0.2717 $\pm$ 0.0692 [0.167, 0.367]
GraphResGNN	0.7071 $\pm$ 0.0402 [0.655, 0.749]	0.7275 $\pm$ 0.0144 [0.711, 0.747]	0.3333 $\pm$ 0.1000 [0.150, 0.425]
GraphCrossGNN	0.7017 $\pm$ 0.0520 [0.608, 0.757]	0.7207 $\pm$ 0.0322 [0.694, 0.784]	0.2850 $\pm$ 0.0868 [0.167, 0.408]

表 5: 补充鲁棒性 benchmark 结果。单元格为五折 mean $\pm$ std [min, max]。加粗表示该数据集上的最佳均值。

模型	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7343 $\pm$ 0.0433 [0.684, 0.811]	0.8295 $\pm$ 0.0456 [0.755, 0.877]	0.7830 $\pm$ 0.0170 [0.764, 0.812]
NodeResGNN	0.7451 $\pm$ 0.0504 [0.684, 0.838]	0.8615 $\pm$ 0.0314 [0.800, 0.885]	0.7874 $\pm$ 0.0142 [0.769, 0.809]
NodeCrossGNN	0.7398 $\pm$ 0.0523 [0.684, 0.838]	0.8850 $\pm$ 0.0380 [0.818, 0.930]	0.7939 $\pm$ 0.0196 [0.770, 0.826]
GraphResGNN	0.7238 $\pm$ 0.0778 [0.649, 0.865]	0.9090 $\pm$ 0.0194 [0.890, 0.935]	0.8022 $\pm$ 0.0244 [0.763, 0.832]
GraphCrossGNN	0.7397 $\pm$ 0.0531 [0.684, 0.838]	0.8740 $\pm$ 0.0460 [0.800, 0.943]	0.7992 $\pm$ 0.0232 [0.756, 0.826]

只有 0.3333，且该任务是六分类。因此 ENZYMES 更适合被解释为支持性的难度压力测试，而非主要证据。

3.3 补充鲁棒性参考

补充鲁棒性数据集的结果见 Appendix A。其内部排序依然混合，例如 NodeResGNN 在 MUTAG 上最优，而 GraphResGNN 在 AIDS 和 Mutagenicity 上最优。这对验证结论的鲁棒性有帮助，但这些数据集不定义本文的中心 biological claim。

图 5 说明了对于交叉残差交换，哪些说法可以成立，哪些不能。最优交叉残差变体在六个数据集中有五个优于 PlainGNN，最大增益出现在 ENZYMES (+0.057)、AIDS (+0.028) 和 Mutagenicity (+0.022)。然而，最优交叉残差变体只在 MUTAG (+0.005) 和 Mutagenicity (+0.001) 上超过最优残差变体，在 DD 上则基本持平 (-0.001)。因此，交叉残差交换是一种有价值的选项，但并不是对最强残差基线的普适替代，包括在蛋白质导向核心内部也是如此。

3.4 机制：表示连续性与性能的张力

为什么 GraphResGNN 明确地在块间重塑表示，却反而优于那些更忠实保留表示的架构？为回答这一问题，我们在 PROTEINS 的 GCNConv 设定下，从 $L = 1$ 到 $L = 8$ 跟踪隐藏状态相似度（CKA 与 cosine）和逐层梯度范数。

图 6 揭示了一个反直觉模式。在 $L = 8$ 时，CKA 排序为 NodeResGNN (0.968) > GraphCrossGNN (0.952) > PlainGNN (0.951) > NodeCrossGNN (0.946) > GraphResGNN (0.942)。最忠实保留表示的架构 NodeResGNN 只在 24 个组合中获胜 3 次；而最积极重

表 6: 各数据集优胜模型摘要, 并给出最大/最小折准确率。

数据集	优胜模型	平均准确率	最大准确率	最小准确率
PROTEINS	NodeResGNN	0.7143	0.7444	0.6847
DD	GraphResGNN	0.7275	0.7468	0.7106
ENZYMES	GraphResGNN	0.3333	0.4250	0.1500
MUTAG	NodeResGNN	0.7451	0.8378	0.6842
AIDS	GraphResGNN	0.9090	0.9350	0.8900
Mutagenicity	GraphResGNN	0.8022	0.8316	0.7629

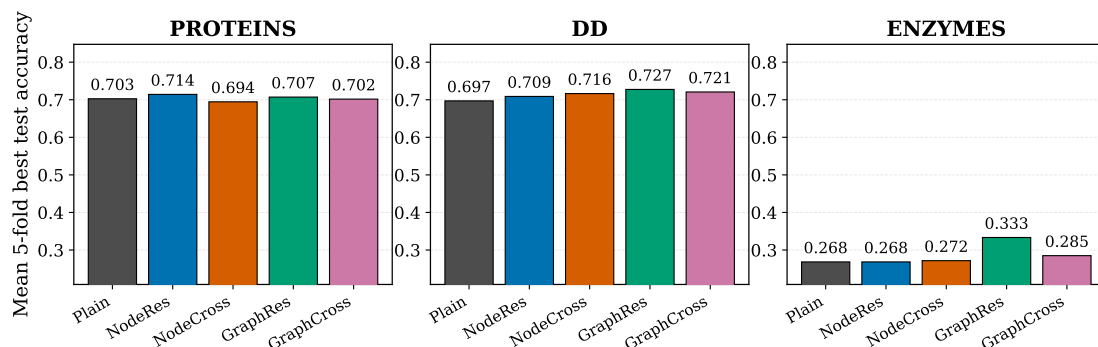


图 4: 蛋白质导向核心上的 V3 结果。主要比较由 PROTEINS 和 DD 承载, ENZYMES 作为支持性的压力测试保留。

塑表示的 GraphResGNN 则在 24 个组合中获胜 12 次。我们将这一现象称为 **CKA-性能悖论**: 更强的层间表示连续性并不能预测更好的最终准确率。cosine 结果呈现相同排序, 为简洁起见不在正文中展开。

这一悖论有其结构性解释。GraphResGNN 的三个图条件块在每个阶段都接收上一块的池化图嵌入作为条件信号。正是这种有意识的、结构化的重塑, 即在每个阶段重新注入逐步累积的全局上下文, 使得最终表示更具判别性。相反, NodeResGNN 对表示的忠实保留, 可能限制了模型对任务特定结构的适应能力。

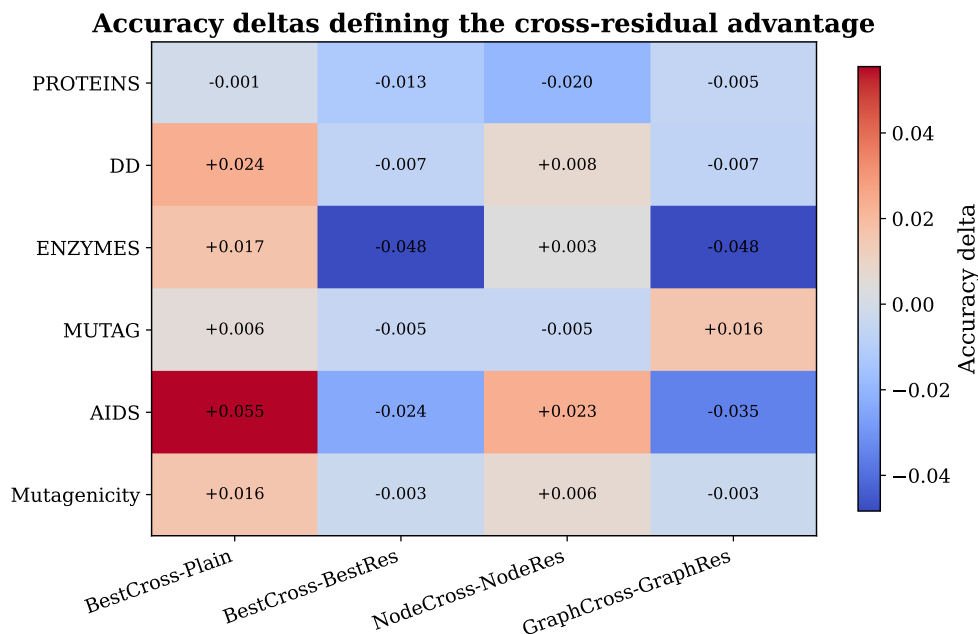


图 5: V3 交叉残差优势热力图。正值单元表示该数据集上最优交叉残差变体优于对应比较基线。

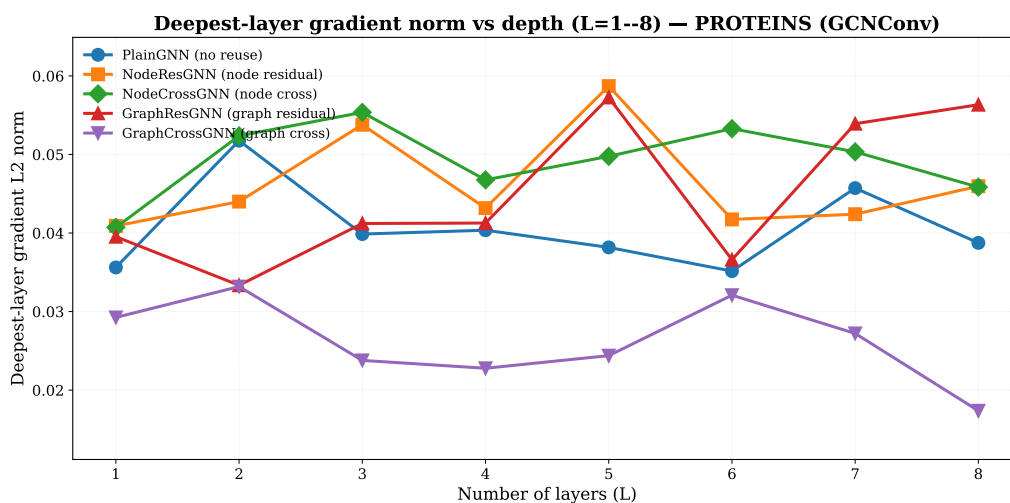


图 7: 最深层梯度的 L_2 范数随 $L = 1-8$ 的变化 (PROTEINS, GCNConv)。

图 7 进一步表明, PlainGNN 的最深层梯度范数随深度下降最快; 而残差家族整体保持了更稳定的梯度。NodeResGNN 与 NodeCrossGNN 的梯度稳定性相近, 这说明跨分支交换并未引入明显的梯度干扰。

3.5 量化这些权衡

为避免只停留在定性描述, 表 7 统计了全部 24 个数据集-算子组合中各架构的获胜次数, 表 8 则衡量跨数据集的排序稳定性。

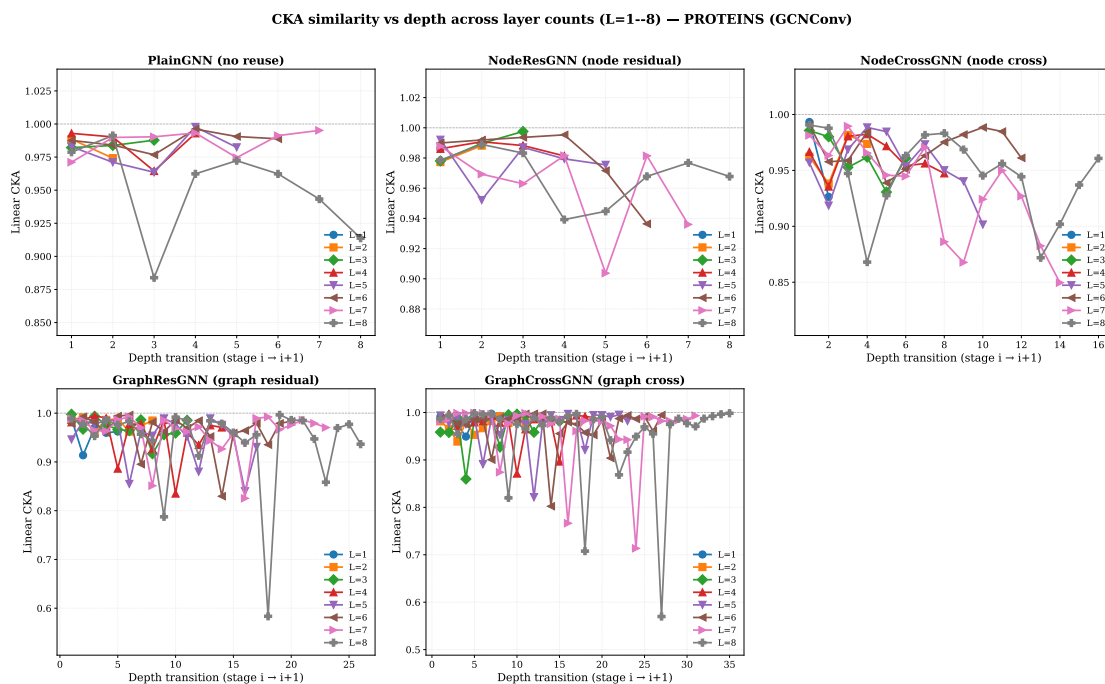


图 6: 在 PROTEINS 的 GCNConv 设定下, $L = 1-8$ 不同深度阶段之间的 CKA 相似度。

表 7: 24 个数据集-算子组合上的获胜次数统计 (CR-GNN 家族)。

模型	PROTEINS	DD	ENZYMES	MUTAG	AIDS	Mutagenicity	总计
GraphResGNN	3	1	3	0	3	2	12
NodeCrossGNN	0	3	1	0	1	1	6
NodeResGNN	1	0	0	1	0	1	3
PlainGNN	0	0	0	2	0	0	2
GraphCrossGNN	0	0	0	1	0	0	1

这些权衡现在可以被定量表述。GraphResGNN 在 24 个数据集-算子组合中获胜最多 (12/24)，但排序稳定性较弱 (std 1.46)，它既可能明显领先，也可能像在 MUTAG 上那样跌至末位。GraphCrossGNN 只获胜 1 次，但它与 PlainGNN 并列拥有最低的排序标准差 (0.75)，同时整体排名更靠前。NodeCrossGNN 则处于中间地带：获胜数第二多 (6/24)，稳定性中等 (std 1.00)，并在 DD 的算子级比较中表现出明显集中优势。

3.6 消融证据

3.6.1 Gate ablation

gate ablation 用于检验前述结构差异是否主要由残差注入的自适应控制所解释。在最强的图级残差目标 (AIDS + GraphResGNN + GINConv) 上，可学习门控表现最佳，但固定系数 0.5 也很接近。在六个偏向交叉残差的目标中，可学习门控和固定门控各获胜三次，说明交叉残差架构对 gate 配置较为稳健，这与其更稳定的整体排名特征是一致

表 8: 六个数据集上的平均排名与排序稳定性 (GCNConv)。

模型	平均排名	距最优差距	排名标准差
GraphResGNN	1.8	0.005	1.46
GraphCrossGNN	2.7	0.018	0.75
NodeCrossGNN	3.0	0.022	1.00
NodeResGNN	3.2	0.024	1.57
PlainGNN	4.3	0.036	0.75

表 9: GraphResGNN + GINConv 在 AIDS 上的 gate ablation。数值为五折平均准确率 $\pm$ 标准差。加粗表示该列最佳。

Gate 设置	平均准确率
Learnable	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9605 $\pm$ 0.0144
Fixed 1.0	0.9480 $\pm$ 0.0300

的。

3.6.2 残差路由

在四个代表性目标上，我们比较了可学习稠密、top-k 与稀疏三种路由。稀疏路由 (阈值 0.05) 在四个目标中赢下三个；只有在最强的图级残差目标上，可学习稠密路由仍然最优。这三个偏向稀疏的目标中有两个涉及交叉残差架构，说明稀疏过滤与跨分支交换更像是互补关系，而非普适压制其他策略的单一解。

3.6.3 统计检验

针对主 biological evidence 的配对检验矩阵 (表 14) 表明，多数架构间差距相对于折级波动而言并不大。对应的补充数据集检验结果见 Appendix A。

3.6.4 参数扫描

对额外 top-k 比例和稀疏阈值的扫描只是细化了上述模式，而没有改变其方向。最优 top-k 比例随目标而变化，而过强稀疏 (0.10) 会损害图级残差目标，这说明稀疏收益依赖阈值设置，而不是无条件成立。

3.7 探索性敏感性分析

在 PROTEINS、DD 和 ENZYMES 上围绕深度、dropout 和学习率所做的辅助敏感性检验，并未实质改变上述主要结论。

表 10: AIDS + GraphResGNN + GINConv 的折级 gate ablation。加粗表示该列最佳。

Gate 设置	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Mean $\pm$ Std
Learnable	0.9500	0.9700	0.9700	0.9675	0.9525	0.9620 $\pm$ 0.0089
Fixed 0.0	0.9225	0.9025	0.9125	0.9000	0.9375	0.9150 $\pm$ 0.0138
Fixed 0.5	0.9325	0.9625	0.9675	0.9675	0.9725	0.9605 $\pm$ 0.0144
Fixed 1.0	0.8925	0.9450	0.9700	0.9775	0.9550	0.9480 $\pm$ 0.0300

表 11: 六个偏向交叉残差目标上的 cross-gate ablation。数值为五折平均准确率 $\pm$ 标准差。加粗表示该行最佳。

数据集	目标	Learnable	Fixed 0.0	Fixed 0.5	Fixed 1.0
AIDS	NodeCross + GATConv	0.9045 $\pm$ 0.0165	0.9075 $\pm$ 0.0065	0.9195 $\pm$ 0.0114	0.9105 $\pm$ 0.0241
DD	NodeCross + GATConv	0.7173 $\pm$ 0.0271	0.7053 $\pm$ 0.0361	0.7156 $\pm$ 0.0225	0.7147 $\pm$ 0.0288
DD	NodeCross + GINConv	0.7071 $\pm$ 0.0306	0.7012 $\pm$ 0.0198	0.7232 $\pm$ 0.0207	0.7113 $\pm$ 0.0277
ENZYMES	NodeCross + GATConv	0.2800 $\pm$ 0.0629	0.2850 $\pm$ 0.0627	0.2550 $\pm$ 0.0674	0.2650 $\pm$ 0.0690
MUTAG	GraphCross + GATConv	0.7558 $\pm$ 0.0495	0.7451 $\pm$ 0.0407	0.7450 $\pm$ 0.0416	0.7504 $\pm$ 0.0779
Mutagenicity	NodeCross + GATConv	0.8114 $\pm$ 0.0110	0.8033 $\pm$ 0.0192	0.8010 $\pm$ 0.0130	0.8036 $\pm$ 0.0078

4 讨论

本节回到引言中提出的三个研究问题，并结合第 3 节证据作答。

4.1 RQ1: 为什么表示连续性最弱的架构反而最常获胜？

CKA-性能悖论，即 GraphResGNN 在深层拥有最低 CKA (0.942) 却赢得最多组合 (12/24)，而 NodeResGNN 拥有最高 CKA (0.968) 却只赢得 3 次，具有明确的结构解释。GraphResGNN 串联三个图条件块，每个块都接收前一块的池化图嵌入作为条件输入。该设计并不追求保留表示，而是利用逐步累积的全局上下文，在每个阶段刻意重塑表示。在当前 benchmark 中，这一策略在 DD、ENZYMES、AIDS 和 Mutagenicity 上更强，但并非在所有地方都最佳。

NodeResGNN 更忠实地保留表示，但也存在代价：如果表示在层间变化过小，模型可能无法充分适应任务特定结构。交叉残差架构在一定程度上缓解了这种张力，因为它为单分支可能压制掉的信息提供了另一条路径，却不会像 GraphResGNN 那样强制进行激进重塑。这也是 GraphCrossGNN 虽然很少拿到第一，但经常保持在前列的原因。

回答：在本文设定中，积极的表示重塑可以是有益的，但其收益依赖于数据集。更高的连续性不足以保证更好的准确率，而交叉残差交换更应被理解作为一种稳定化替代方案，而不是证明“连续性应当总是降低”。

4.2 RQ1 & RQ2: 不同复用拓扑何时占优？

该 benchmark 支持的是分裂型优势区间，而不是单一赢家。在面向论文主线的 biological comparison 中，NodeResGNN 在 PROTEINS 上最强，而 GraphResGNN 在 DD

表 12: 四个代表性目标上的残差路由与参数扫描摘要（转置形式）。数值为五折平均准确率 $\pm$ 标准差。加粗表示该列最佳。

路由方式	AIDS+GraphRes+GIN	PROT+GraphRes+SAGE	AIDS+NodeCross+GAT	DD+NodeCross+GAT
Learnable	0.9500 $\pm$ 0.0087	0.7178 $\pm$ 0.0411	0.8970 $\pm$ 0.0491	0.7080 $\pm$ 0.0172
Top-k 0.25	0.9075 $\pm$ 0.0237	0.7241 $\pm$ 0.0407	0.9160 $\pm$ 0.0128	0.7062 $\pm$ 0.0249
Top-k 0.50	0.9330 $\pm$ 0.0251	0.7151 $\pm$ 0.0397	0.8960 $\pm$ 0.0483	0.7029 $\pm$ 0.0160
Top-k 0.75	0.9240 $\pm$ 0.0287	0.7214 $\pm$ 0.0318	0.8960 $\pm$ 0.0489	0.7164 $\pm$ 0.0248
Sparse 0.02	0.9470 $\pm$ 0.0181	0.7160 $\pm$ 0.0248	0.9060 $\pm$ 0.0268	0.7147 $\pm$ 0.0259
Sparse 0.05	0.9250 $\pm$ 0.0152	0.7259 $\pm$ 0.0475	0.9180 $\pm$ 0.0162	0.7181 $\pm$ 0.0196
Sparse 0.10	0.8315 $\pm$ 0.0549	0.7088 $\pm$ 0.0521	0.9025 $\pm$ 0.0139	0.7105 $\pm$ 0.0243
最佳	Learnable	Sparse 0.05	Sparse 0.05	Sparse 0.05

表 13: 不同架构家族上的路由优胜摘要。

目标	最佳 learnable	最佳 top-k	最佳 sparse	总体最优
AIDS + GraphRes + GIN	0.9500	0.9330	0.9470	Learnable
PROTEINS + GraphRes + SAGE	0.7178	0.7241	0.7259	Sparse
AIDS + NodeCross + GAT	0.8970	0.9160	0.9180	Sparse
DD + NodeCross + GAT	0.7080	0.7164	0.7181	Sparse

上最强。交叉残差变体在 DD 上最有说服力：在算子级 winner count 中，NodeCrossGNN 赢下四个 DD 算子设定中的三个，尽管在单一 GCNConv 主表中，GraphResGNN 仍是最优 DD 模型。这个区别很重要，因为它表明 DD 上的交叉残差优势在跨算子层面是稳的，但又不能被简化为某一次单模型 sweep。ENZYMES 也指向 GraphResGNN 更强，但由于所有模型在该数据集上都表现较弱，它更适合被解释为压力测试，而不是与主证据并列。

在图级摘要更有用的任务上，GraphResGNN 是更强的默认选择；在最优残差基线已能匹配或超过最优交叉残差变体的数据集上，交叉交换应被视为有竞争力的备选，而不是默认首选。图 5 对这一边界做了明确说明：最优交叉变体通常优于 PlainGNN，但很少优于最优残差模型。补充数据集有助于检验这一边界并非蛋白质核心独有，但它们不应反过来重写本文的主 biological argument。

回答：在当前 benchmark 中，图级残差传播可作为最强的默认家族；当任务更接近 PROTEINS 时，节点级残差复用仍是更强选择；而交叉残差交换是一种选择性方案，尤其值得在类似 DD 的蛋白质图和跨算子比较中测试。

4.3 RQ3: 路由策略是否在架构选择之外仍然重要？

第 3 节中的消融表明答案是肯定的，而且这种作用依赖目标。第一，gate ablation 显示交叉残差架构对 gate 配置较为稳健，可学习与固定 gate 在偏向交叉的目标上各赢三次，这与其稳定的排序画像一致。第二，路由模式确实重要：稀疏路由（阈值 0.05）在四个代表性目标中赢下三个，其中两个涉及交叉残差架构。其机制层面的协同在于，稀

表 14: 主 biological benchmark 数据集上的配对 t 检验。单元格内容为 Δ (Cohen's d , p)。加粗表示 $p < 0.05$ ($n=5$ folds)。

数据集	比较	GCNConv	GATConv	SAGEConv	GINConv
PROTEINS	NodeRes – Plain	+0.012 (+0.55, 0.286)	+0.000 (+0.00, 0.999)	+0.006 (+0.31, 0.524)	+0.012 (+0.56, 0.277)
	NodeCross – Plain	-0.008 (-0.28, 0.571)	-0.011 (-0.22, 0.647)	-0.002 (-0.07, 0.881)	-0.008 (-0.23, 0.632)
	GraphRes – Plain	+0.004 (+0.45, 0.375)	+0.015 (+0.48, 0.345)	+0.030 (+1.69, 0.019)	+0.040 (+1.10, 0.069)
	GraphCross – Plain	-0.001 (-0.02, 0.961)	-0.008 (-0.24, 0.623)	+0.004 (+0.24, 0.622)	+0.004 (+0.16, 0.743)
	NodeCross – NodeRes	-0.020 (-0.53, 0.302)	-0.011 (-0.19, 0.690)	-0.008 (-0.72, 0.181)	-0.020 (-0.87, 0.125)
	GraphCross – GraphRes	-0.005 (-0.15, 0.751)	-0.023 (-2.18, 0.008)	-0.026 (-1.47, 0.031)	-0.036 (-2.36, 0.006)
	GraphRes – NodeRes	-0.007 (-0.26, 0.588)	+0.015 (+0.37, 0.459)	+0.023 (+0.75, 0.171)	+0.028 (+1.05, 0.079)
	NodeCross – GraphCross	-0.007 (-0.49, 0.337)	-0.003 (-0.10, 0.838)	-0.005 (-0.21, 0.659)	-0.012 (-0.39, 0.427)
DD	NodeRes – Plain	+0.012 (+0.39, 0.429)	+0.002 (+0.24, 0.624)	+0.016 (+0.48, 0.347)	+0.001 (+0.03, 0.946)
	NodeCross – Plain	+0.019 (+0.55, 0.286)	+0.035 (+0.90, 0.115)	+0.023 (+0.87, 0.124)	+0.025 (+0.93, 0.107)
	GraphRes – Plain	+0.030 (+0.64, 0.223)	+0.020 (+0.50, 0.326)	+0.021 (+0.91, 0.112)	+0.002 (+0.06, 0.892)
	GraphCross – Plain	+0.024 (+0.87, 0.123)	+0.019 (+0.46, 0.357)	+0.004 (+0.19, 0.699)	+0.008 (+0.20, 0.672)
	NodeCross – NodeRes	+0.008 (+0.43, 0.395)	+0.033 (+0.88, 0.121)	+0.007 (+0.64, 0.226)	+0.024 (+0.94, 0.103)
	GraphCross – GraphRes	-0.007 (-0.23, 0.637)	-0.002 (-0.11, 0.815)	-0.017 (-2.82, 0.003)	+0.006 (+0.14, 0.762)
	GraphRes – NodeRes	+0.019 (+0.64, 0.228)	+0.019 (+0.46, 0.359)	+0.005 (+0.24, 0.624)	+0.001 (+0.03, 0.954)
	NodeCross – GraphCross	-0.004 (-0.31, 0.526)	+0.016 (+1.07, 0.075)	+0.019 (+1.53, 0.027)	+0.017 (+0.57, 0.274)
ENZYMES	NodeRes – Plain	-0.000 (-0.00, 1.000)	+0.005 (+0.19, 0.697)	-0.017 (-0.94, 0.103)	+0.030 (+0.47, 0.351)
	NodeCross – Plain	+0.003 (+0.09, 0.847)	+0.032 (+0.45, 0.369)	+0.007 (+0.24, 0.614)	+0.032 (+2.12, 0.009)
	GraphRes – Plain	+0.020 (+0.92, 0.107)	+0.020 (+0.32, 0.507)	+0.020 (+0.47, 0.360)	+0.068 (+2.45, 0.002)
	GraphCross – Plain	+0.020 (+0.55, 0.280)	+0.007 (+0.08, 0.875)	+0.020 (+0.33, 0.502)	+0.028 (+0.52, 0.303)
	NodeCross – NodeRes	+0.003 (+0.08, 0.878)	+0.027 (+0.36, 0.471)	+0.023 (+0.42, 0.399)	+0.002 (+0.07, 0.884)
	GraphCross – GraphRes	-0.000 (-0.01, 0.981)	-0.013 (-0.21, 0.672)	+0.000 (+0.00, 1.000)	-0.040 (-0.62, 0.237)
	GraphRes – NodeRes	+0.020 (+0.57, 0.264)	+0.015 (+0.28, 0.562)	+0.037 (+0.82, 0.151)	+0.038 (+1.02, 0.079)
	NodeCross – GraphCross	-0.017 (-0.47, 0.355)	+0.025 (+0.68, 0.211)	-0.013 (-0.40, 0.444)	+0.003 (+0.07, 0.894)

疏性能够在残差信号跨深度累积噪声前先抑制弱通道，而跨分支路径则保留真正互补的信息。过强稀疏 (0.10) 会损害图级残差目标，这再次说明其作用依赖阈值，而不是普适定律。

回答：路由策略是一个有意义的次级设计轴。轻度稀疏路由通常是交叉残差交换的更佳搭档，而可学习稠密路由则仍在最强的图级残差目标上保持优势。

4.4 局限性

这些结论的适用范围受到若干限制。首先，统计效能有限：在仅有五个折的前提下，只有较大效应才容易达到 $p < 0.05$ 。其次，CKA-性能悖论目前仅在单一数据集-算子组合 (PROTEINS, GCNConv) 上被系统记录，其能否推广仍待验证。第三，这些 benchmark 数据集属于中等规模生物分子图，在更大规模或不同类型图上，权衡关系可能改变。第四，路由分析只覆盖四个目标，足以揭示模式，但还不足以支撑普适性结论。

4.5 未来方向

最重要的扩展方向，是在更大 benchmark 和更丰富输入上继续检验 CKA-性能悖论。如果该现象能够泛化，它将改变我们评价深层 GNN 的方式，把关注点从层间稳定性指标转向表示变化是否具有结构性。稀疏路由与交叉残差的协同关系，也应在超过当前四个目标的更广泛设定中继续测试。

5 结论

本文在受控图分类协议下比较了五种残差路由拓扑，发现峰值准确率、跨数据集一致性和表示连续性之间存在系统性权衡。本文最核心的贡献，是在不改变主干、池化规则和评估协议的前提下，构建出一个能将这些权衡定量呈现出来的比较框架。

回到三个研究问题：(1) 最强架构取决于数据集区间，而不是单一普适的残差规则；在面向论文主线的蛋白质导向比较中，NodeResGNN 在 PROTEINS 上最优，而 GraphResGNN 在 DD 上最优。ENZYMES 指向相同的 GraphResGNN 方向，但主要作为支持性压力测试，而非与主证据并列。(2) 交叉残差交换有用，但具有选择性：它通常优于 PlainGNN，在 DD 和 Mutagenicity 上保持竞争力，并在 DD 的算子级比较中尤为值得关注，但它并不会稳定超过最优残差基线。(3) 路由策略是有意义的次级设计轴：在针对性消融中，轻度稀疏路由最常最优，而可学习稠密路由仍在最强的图级残差目标上保持最佳。

在 PROTEINS(GCNConv)上的深度扩展分析揭示了一个值得进一步验证的“CKA-性能悖论”：表示连续性最高的 NodeResGNN 并未最常获胜，而表示连续性最低的 GraphResGNN 却在更广泛的数据集-算子 sweep 中获胜最多。这一现象提示，但尚未最终证明，层间的结构化表示变化可能是有益而非有害的。

本文最强的结论由以下证据共同支撑：蛋白质导向核心上的分层五折 benchmark、针对性的五折消融以及 $L = 1-8$ 的深度扩展分析。本文解决的是中等规模生物分子 benchmark 上的受控图分类行为；未来工作应继续检验 CKA-性能悖论，以及“稀疏路由 + 交叉残差”这一组合，是否能推广到更大图、更丰富的生物输入和超越 benchmark 级别的任务场景。

6 致谢

作者感谢开源图学习社区以及本文所用 benchmark 资源的维护者，也感谢审稿人与编辑投入的时间和反馈意见。

A 附录

A.1 补充鲁棒性 benchmark

本节数据集被保留为蛋白质导向主比较之外的鲁棒性检验。它们有助于观察架构排序在 PROTEINS-DD 核心之外的变化方式，但并不定义本文的主要 biological claim。

A.2 补充统计检验

参考文献

- Uri Alon and Eran Yahav. On the bottleneck of graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2021.
- Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.
- Tianle Cai, Jiacheng Luo, Sheng Wang, Kai Zhang, Yu Tian, Liwei Yang, Zekun Li, and Kilian Weinberger. Graphnorm: A principled approach to accelerating graph neural network training. In *International Conference on Machine Learning*, pages 1277–1287, 2021.
- Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. Simple and deep graph convolutional networks. In *International Conference on Machine Learning (ICML)*, pages 1725–1735, 2020a.
- Yuning Chen, Xihao Wei, Pan Xu, Xinwen Wang, Ping Luo, Zhiyuan Li, and Jian Tang. Revisiting over-smoothing in deep gcns. *arXiv preprint arXiv:2003.13663*, 2020b.
- Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. *arXiv preprint arXiv:1903.02428*, 2019.
- Hongyang Gao and Shuiwang Ji. Graph u-net. *arXiv preprint arXiv:1905.05178*, 2019.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*, pages 1263–1272, 2017.
- William L Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in neural information processing systems*, pages 1024–1034, 2017.

表 15: 主 biological benchmark 结果。数值为 mean best test accuracy $\pm$ 标准差。

模型	PROTEINS	DD	ENZYMES
PlainGNN	0.7053 $\pm$ 0.0271	0.7165 $\pm$ 0.0179	0.2450 $\pm$ 0.0407
NodeResGNN	0.6945 $\pm$ 0.0509	0.7207 $\pm$ 0.0259	0.2650 $\pm$ 0.0661
NodeCrossGNN	0.6963 $\pm$ 0.0433	0.7198 $\pm$ 0.0178	0.3000 $\pm$ 0.0580
GraphResGNN	0.7223 $\pm$ 0.0367	0.6934 $\pm$ 0.0458	0.3117 $\pm$ 0.0746
GraphCrossGNN	0.6864 $\pm$ 0.0465	0.7097 $\pm$ 0.0309	0.3017 $\pm$ 0.0750

表 16: 补充鲁棒性 benchmark 结果。

模型	MUTAG	AIDS	Mutagenicity
PlainGNN	0.7236 $\pm$ 0.0486	0.8455 $\pm$ 0.0257	0.7814 $\pm$ 0.0235
NodeResGNN	0.7343 $\pm$ 0.0734	0.8690 $\pm$ 0.0354	0.7897 $\pm$ 0.0139
NodeCrossGNN	0.7397 $\pm$ 0.0676	0.8735 $\pm$ 0.0412	0.8033 $\pm$ 0.0159
GraphResGNN	0.7290 $\pm$ 0.0585	0.9160 $\pm$ 0.0121	0.8022 $\pm$ 0.0220
GraphCrossGNN	0.7129 $\pm$ 0.0539	0.8710 $\pm$ 0.0412	0.8029 $\pm$ 0.0182

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Johannes Klicpera, Alan Bojchevski, and Stephan Günnemann. Combining label propagation and simple models out-performs graph neural networks. *arXiv preprint arXiv:1810.05997*, 2018.

Guohao Li, Elias Muller, Abbas Thabet, and Bernard Ghanem. Deeper insights into graph convolutional networks for semi-supervised learning. *arXiv preprint arXiv:1809.02584*, 2018.

Christopher Morris, Roger Wattenhofer, and Kristian Kersting. Tudataset: A collection of benchmark datasets for learning with graphs. *arXiv preprint arXiv:2007.08663*, 2020.

Kenta Oono and Taiji Suzuki. Simple and deep graph convolutional networks. *arXiv preprint arXiv:2006.13604*, 2020.

Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

表 17: 各数据集的优胜模型、优化深度与参数量摘要。

数据集	优胜模型	平均准确率	平均最佳 epoch	参数量
PROTEINS	GraphResGNN	0.7223	58.8	51208
DD	NodeResGNN	0.7207	25.4	22530
ENZYMES	GraphResGNN	0.3117	91.4	52248
MUTAG	NodeCrossGNN	0.7397	88.2	34434
AIDS	GraphResGNN	0.9160	83.8	57928
Mutagenicity	NodeCrossGNN	0.8033	131.2	35330

Jake Topping, Francesco Di Giovanni, Benjamin Chamberlain, Michael Bronstein, Douwe Vendrig, and Pietro Lio. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

Zonghan Wu, Shirui Pan, Fengwen Chen, Guoding Long, Cheng Zhang, and Sheng Yu Zhang. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2021.

Keyulu Xu, Chengtao Li, Yonglong Tian, Xiao Du, Jure Leskovec, and Stefanie Jegelka. Representation learning on graphs with jumping knowledge networks. In *International Conference on Machine Learning*, pages 5449–5458, 2018.

Rex Ying, Jiaxuan You, Christopher Morris, Xiang Ren, William L Hamilton, and Jure Leskovec. Hierarchical graph representation learning with differentiable pooling. In *Advances in neural information processing systems*, pages 4800–4810, 2018.

Lingxiao Zhao and Leman Akoglu. Pairnorm: Tackling over-smoothing in gns. *arXiv preprint arXiv:2009.10698*, 2020.

Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhaocheng Liu, Zhongyuan Wang, Peng Li, Ming Sun, Yu Shi, et al. Graph neural networks: A review of methods and applications. *AI Open*, 1:57–81, 2020.

表 18: 补充鲁棒性数据集上的配对 t 检验。单元格内容为 Δ (Cohen’s d , p)。加粗表示 $p < 0.05$ ($n=5$ folds)。

数据集	比较	GCNConv	GATConv	SAGEConv	GINConv
MUTAG	NodeRes – Plain	+0.011 (+0.29, 0.545)	+0.000 (+0.00, 1.000)	-0.016 (-0.40, 0.416)	-0.005 (-0.12, 0.812)
	NodeCross – Plain	+0.016 (+0.36, 0.477)	+0.005 (+0.09, 0.850)	+0.005 (+0.15, 0.758)	+0.016 (+0.34, 0.498)
	GraphRes – Plain	+0.005 (+0.09, 0.864)	+0.011 (+0.33, 0.513)	-0.011 (-0.10, 0.839)	-0.016 (-0.22, 0.670)
	GraphCross – Plain	-0.011 (-0.20, 0.683)	+0.027 (+0.59, 0.262)	-0.022 (-0.28, 0.576)	-0.005 (-0.16, 0.756)
	NodeCross – NodeRes	+0.005 (+0.10, 0.836)	+0.005 (+0.10, 0.845)	+0.022 (+0.33, 0.516)	+0.022 (+0.37, 0.464)
	GraphCross – GraphRes	-0.016 (-0.34, 0.515)	+0.016 (+0.37, 0.467)	-0.011 (-0.26, 0.613)	+0.011 (+0.25, 0.618)
	GraphRes – NodeRes	-0.005 (-0.12, 0.808)	+0.011 (+0.17, 0.742)	+0.005 (+0.08, 0.881)	-0.011 (-0.12, 0.824)
	NodeCross – GraphCross	+0.027 (+0.71, 0.186)	-0.022 (-0.61, 0.240)	+0.027 (+0.53, 0.310)	+0.022 (+0.48, 0.349)
AIDS	NodeRes – Plain	+0.023 (+1.19, 0.062)	+0.010 (+0.89, 0.112)	+0.018 (+0.86, 0.125)	+0.015 (+0.59, 0.261)
	NodeCross – Plain	+0.028 (+1.06, 0.083)	+0.018 (+0.80, 0.144)	+0.020 (+1.52, 0.022)	+0.018 (+0.67, 0.222)
	GraphRes – Plain	+0.070 (+2.41, 0.005)	+0.025 (+1.57, 0.041)	+0.028 (+2.45, 0.004)	+0.048 (+2.47, 0.004)
	GraphCross – Plain	+0.025 (+0.94, 0.089)	+0.013 (+1.10, 0.068)	+0.018 (+1.16, 0.067)	+0.023 (+0.86, 0.124)
	NodeCross – NodeRes	+0.005 (+0.41, 0.420)	+0.008 (+0.52, 0.311)	+0.003 (+0.14, 0.778)	+0.003 (+1.61, 0.005)
	GraphCross – GraphRes	-0.045 (-1.46, 0.043)	-0.013 (-0.66, 0.218)	-0.010 (-0.44, 0.398)	-0.025 (-0.87, 0.125)
	GraphRes – NodeRes	+0.047 (+2.62, 0.002)	+0.015 (+1.19, 0.063)	+0.010 (+0.46, 0.370)	+0.033 (+2.02, 0.014)
	NodeCross – GraphCross	-0.003 (-0.10, 0.848)	+0.005 (+0.44, 0.387)	+0.003 (+0.20, 0.682)	-0.005 (-0.16, 0.751)
Mutagenicity	NodeRes – Plain	+0.004 (+0.26, 0.597)	+0.011 (+0.95, 0.101)	+0.014 (+0.93, 0.106)	+0.013 (+0.83, 0.137)
	NodeCross – Plain	+0.011 (+0.68, 0.201)	+0.020 (+1.05, 0.079)	+0.013 (+0.86, 0.127)	+0.017 (+0.95, 0.101)
	GraphRes – Plain	+0.019 (+1.24, 0.050)	+0.008 (+0.57, 0.273)	+0.012 (+0.62, 0.237)	+0.018 (+0.83, 0.139)
	GraphCross – Plain	+0.016 (+0.85, 0.131)	+0.009 (+0.46, 0.366)	+0.012 (+0.49, 0.333)	+0.010 (+0.80, 0.147)
	NodeCross – NodeRes	+0.006 (+0.26, 0.588)	+0.009 (+0.69, 0.195)	-0.001 (-0.13, 0.787)	+0.003 (+0.33, 0.503)
	GraphCross – GraphRes	-0.003 (-0.30, 0.545)	+0.002 (+0.08, 0.874)	+0.000 (+0.00, 0.999)	-0.008 (-0.32, 0.513)
	GraphRes – NodeRes	+0.015 (+0.87, 0.125)	-0.003 (-0.27, 0.583)	-0.003 (-0.18, 0.714)	+0.005 (+0.23, 0.637)
	NodeCross – GraphCross	-0.005 (-0.35, 0.476)	+0.010 (+0.50, 0.324)	+0.001 (+0.10, 0.832)	+0.007 (+0.49, 0.334)